

AIML CA1

Classification & Regression

Moiz | P2508625

Classification

Feature Name	Description
Unique ID	A unique identifier assigned to each machine record.
Product ID	The product code associated with each manufacturing instance.
Quality	Categorical measure of product quality with three levels — L (Low), M (Medium), and H (High).
Ambient T (°C)	The ambient temperature in degrees Celsius surrounding the machine during operation.
Process T (°C)	The process temperature in degrees Celsius within the machinery during production.
Rotation Speed (rpm)	The machine's rotation speed measured in revolutions per minute (rpm) .
Torque (Nm)	The torque produced by the machine, measured in Newton-meters (Nm) .
Tool Wear (min)	The total tool usage time before replacement, recorded in minutes .
Machine Status	The target variable indicating machine condition: 0 = normal operation, 1 = faulty/abnormal state.

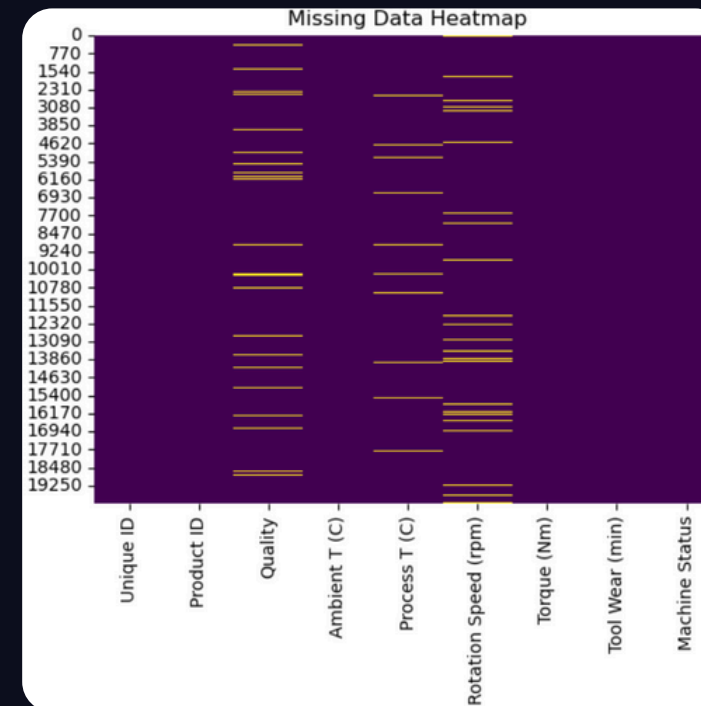
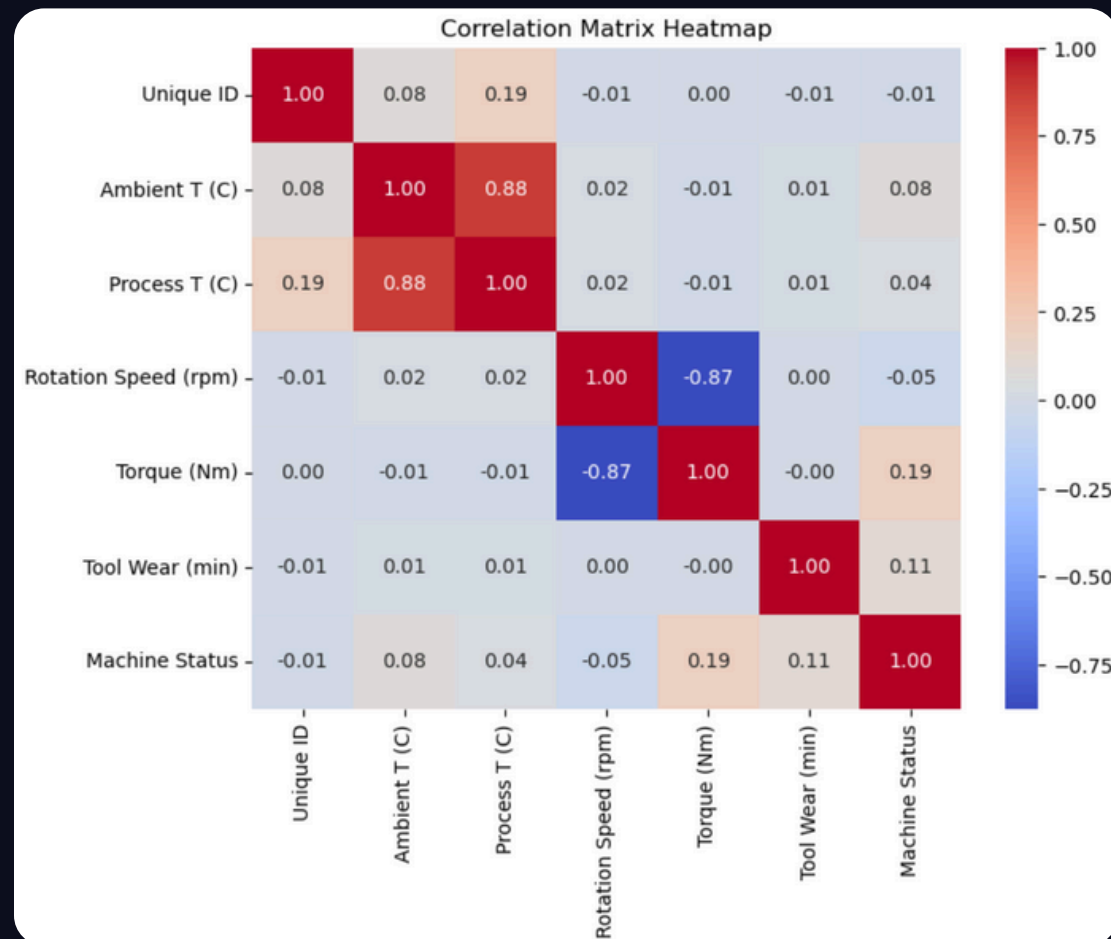
← Target Variable → Working / Faulty

Data Overview

factory_data.csv contains **20,000** Rows and **9** Attributes, comprising as a mix of **Categorical** and **Numerical** features.

Objective: Build a **Recall-optimised** model that effectively identifies faulty machines.

EDA Highlights



- **Feature Correlations**

- **Torque** ↔ **Rotation Speed** show a strong *negative* relationship.
- **Ambient Temperature** ↔ **Process Temperature** are also strongly correlated.

- **Missing Values**

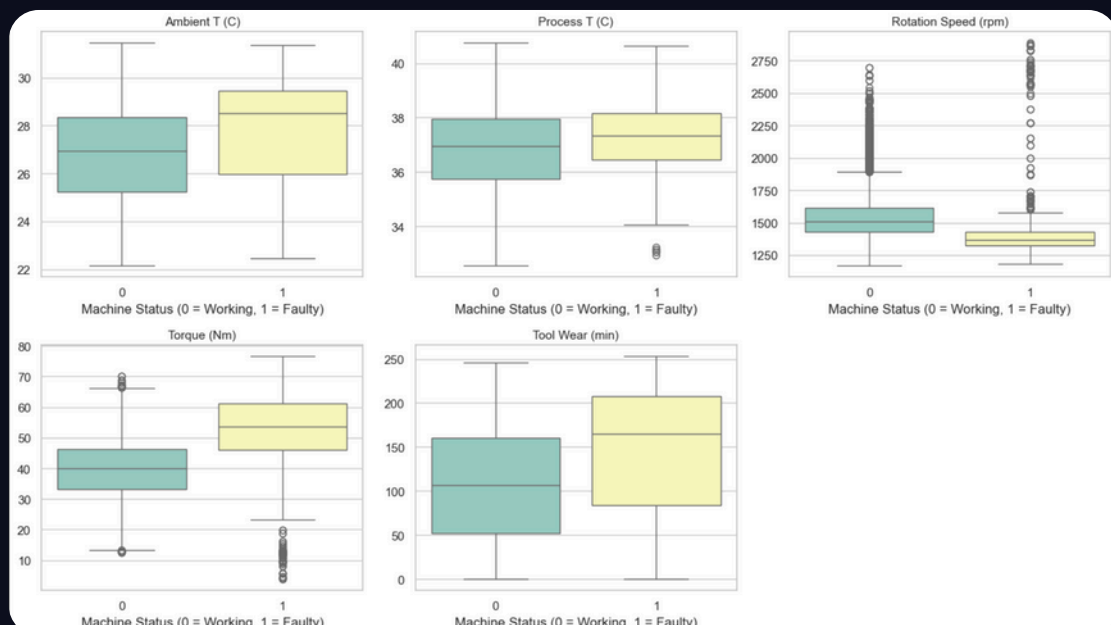
- Only small amounts of missing data.
- Affected features: **Quality**, **Process Temperature**, **Rotation Speed**. (991, 400 and 1188, respectively)
- Confirmed through the Missing Data Heatmap.

- **Working vs Faulty Machine Comparison**

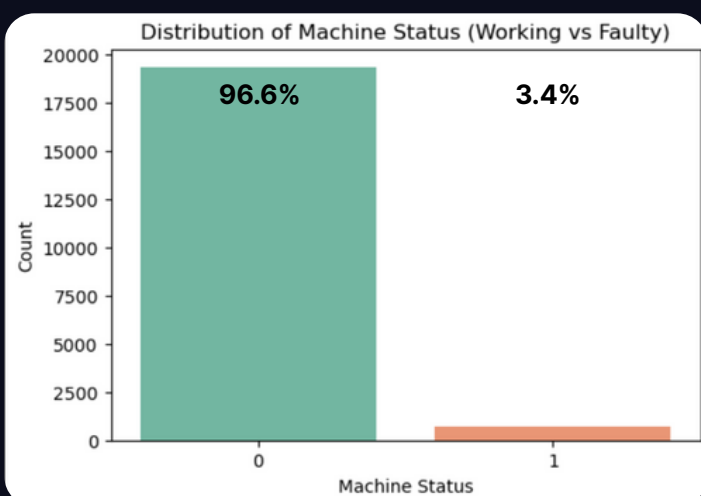
- **Torque**: noticeably higher in faulty machines.
- **Tool Wear**: significantly increased in faulty cases.
- **Rotation Speed**: decreases when machines are faulty.
- **Temperatures (ambient & process)**: show slight increases in faulty machines.

- **Target Imbalance**

- Dataset is moderately imbalanced.
- Ratio of working : faulty ≈ **29 : 1**.



Machine Status	0	1	Difference
Tool Wear (min)	106.694	143.782	37.088
Torque (Nm)	39.630	50.168	10.538
Ambient T (C)	26.874	27.786	0.912
Process T (C)	36.894	37.190	0.296
Rotation Speed (rpm)	1540.330	1495.300	-45.030



Data Preprocessing + SMOTE

- Dropped **irrelevant** identifier columns and converted Quality into numeric form using **LabelEncoder**.
- Separated the dataset into **Features (X)** and **Target (Machine Status)**.
- Performed a **stratified** 80–20 train–test split, preserving original class proportions.
- Scaled all feature columns using **MinMaxScaler**, fitted on the training data to **prevent data leakage**.
- Left the target variable unscaled since it represents categorical labels (Working/Faulty).
- Applied **KNN Imputer** fitted only on X_train, then used it to transform both training and test sets.
- Checked final dataset shapes and confirmed completeness across all processed data.

```
# 1. Drop irrelevant columns
df = df.drop(['Unique ID', 'Product ID'], axis=1)

# 2. Encode categorical features
label_encoder = LabelEncoder()
df['Quality'] = label_encoder.fit_transform(df['Quality'].astype(str))

# 3. Separate features and target BEFORE any imputation
X = df.drop('Machine Status', axis=1)
y = df['Machine Status']

# 4. Train-test split FIRST
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 5,6. Pipeline for Scaling and Imputing data
preprocess_pipeline = Pipeline([
    ("scaler", MinMaxScaler()),
    ("imputer", KNNImputer(n_neighbors=5))
])

X_train_processed = preprocess_pipeline.fit_transform(X_train)
X_test_processed = preprocess_pipeline.transform(X_test)
```

Quality	20000
Ambient T (C)	20000
Process T (C)	20000
Rotation Speed (rpm)	20000
Torque (Nm)	20000
Tool Wear (min)	20000

Usage of **SMOTE (Synthetic Minority Over-sampling Technique)**

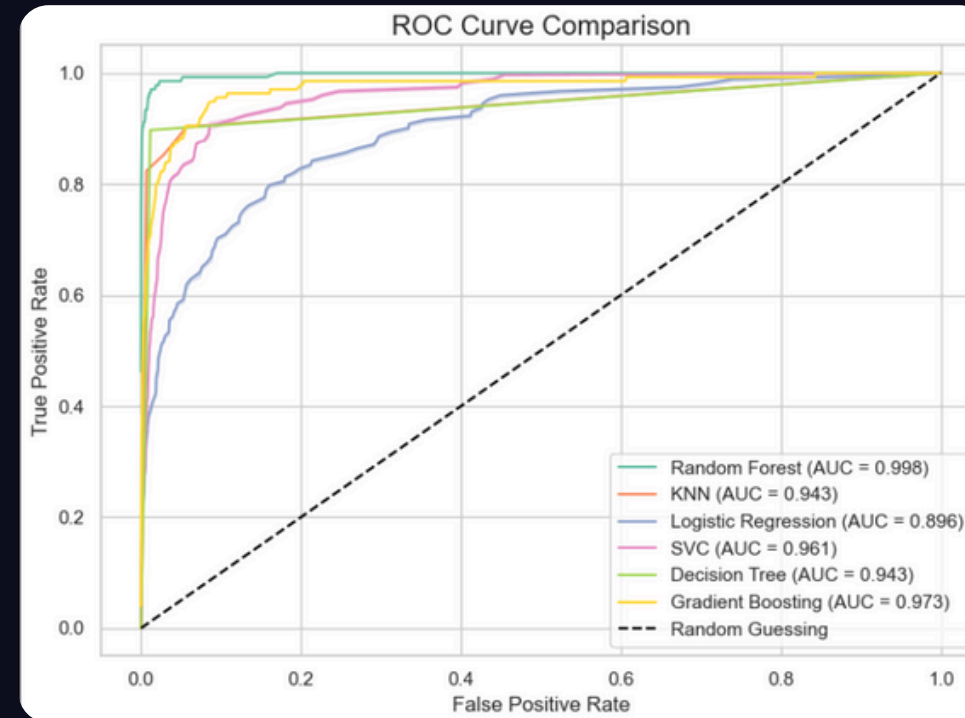
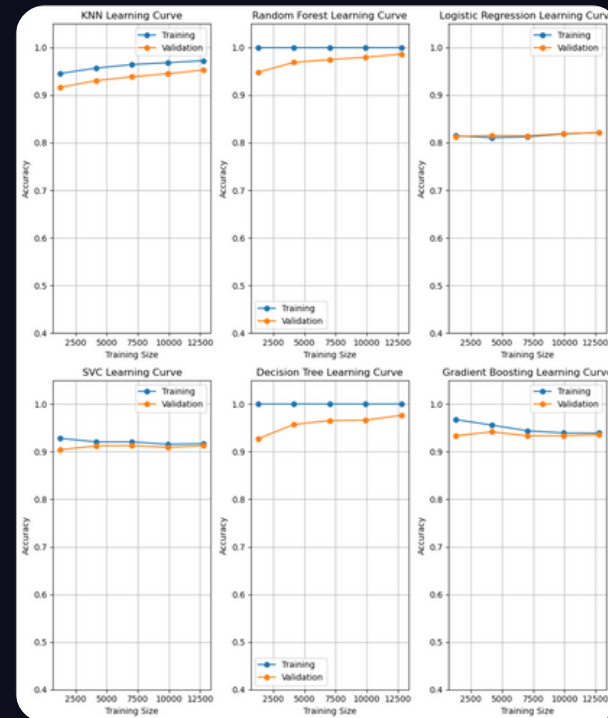
SMOTE was applied on X_train to create synthetic samples for the minority (Faulty) class, resulting in a balanced **50:50** distribution, instead of the **29:1** distribution discovered in EDA. This helps models learn both classes (Working & Faulty) more effectively, reducing **bias** towards the majority class.



Training, Selection & Evaluation

Models choosen for Training

- **Random Forest**
- K Nearest Neighbors (KNN)
- Logistic Regression
- Decision Tree
- **Gradient Boosting Classifier**
- **Support Vector Classifier (SVC)**



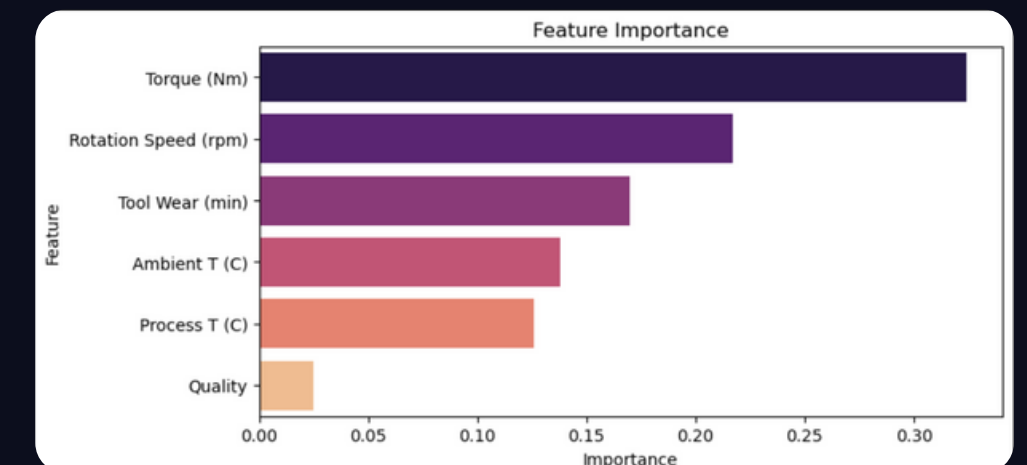
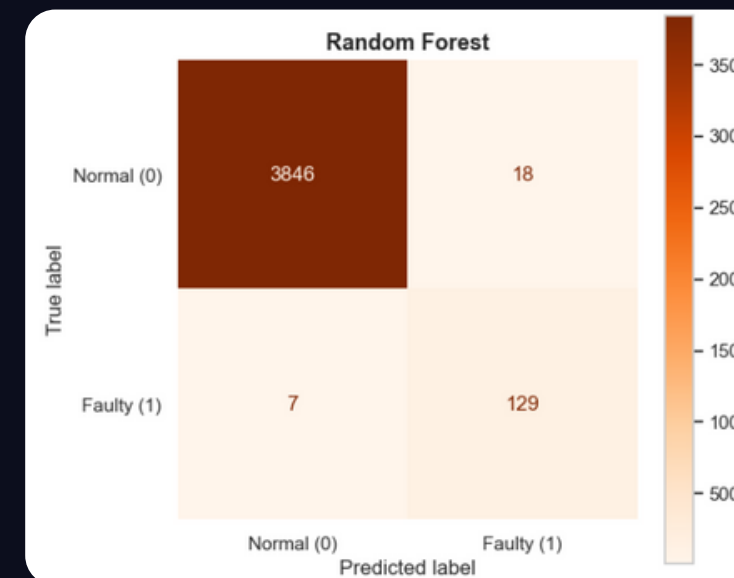
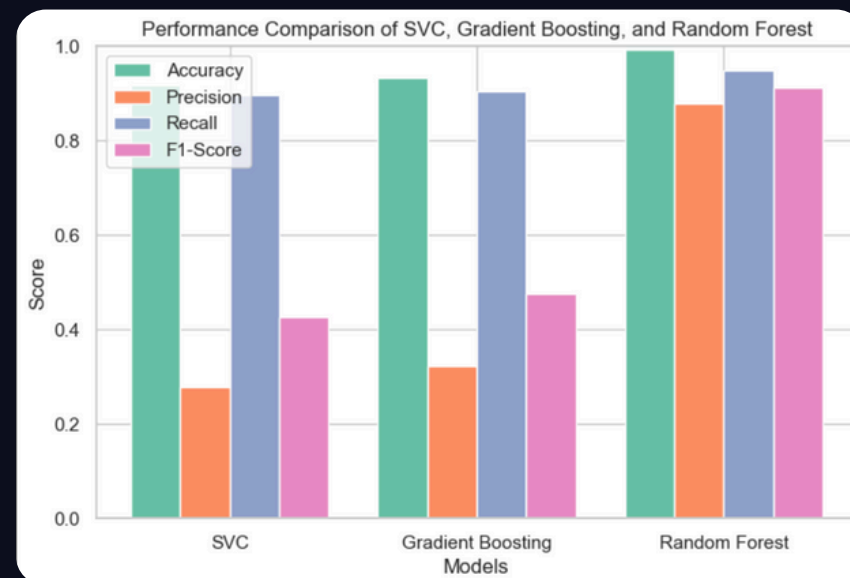
Best Model: **Random Forest**

- AUC: **0.998** → Excellent Generalisation.
- Accuracy: **0.99** → correctly classifies 99% of all instances.
- Precision: **0.86** → 86% of predicted faults are actually faults.
- Recall: **0.92** → 92% of all faults were detected.
- F1-Score: **0.89** → Strong balance between precision and recall.

Training Results



1. **Random Forest**
2. Gradient Boosting Classifier
3. Support Vector Classifier



As seen in EDA, **Torque**, **Rotation Speed** and **Tool Wear** are key predictors of the Faulty Class

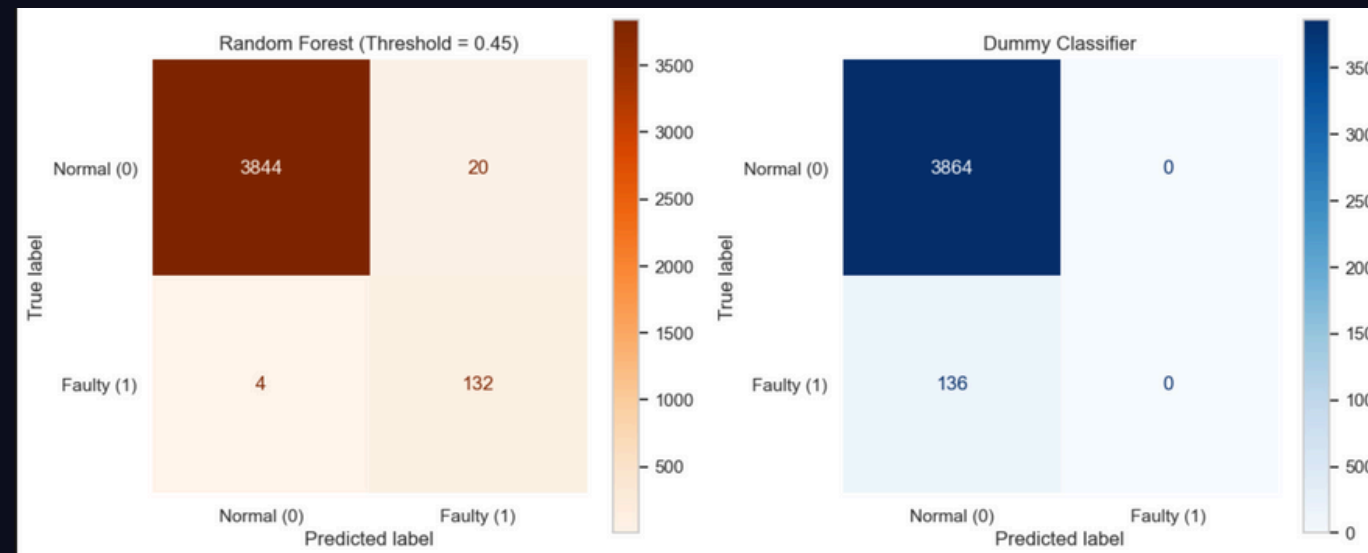
Model Improvement

Hyperparameter Tuning

- Used **GridSearchCV** with 5-fold stratified cross-validation to find **optimal** Random Forest parameters
- Tuned parameters: `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`
- Optimised for **F1-score** and **recall** to balance fault detection and false alarms

Classification Threshold Tuning

- Applied custom threshold of **0.45** (instead of default 0.5) to predicted probabilities
- Lower threshold prioritizes recall → catches more faults at the cost of some false positives
- Optimised balance between detecting critical faults and minimising unnecessary alerts



Tuned Random Forest Classification Report (Threshold = 0.45)

	precision	recall	f1-score	support
Normal (0)	1.00	0.99	1.00	3864
Faulty (1)	0.87	0.97	0.92	136
accuracy			0.99	4000
macro avg	0.93	0.98	0.96	4000
weighted avg	0.99	0.99	0.99	4000

In conclusion, the final optimised model achieves exceptional performance, catching 97% of all faults while keeping false alarms at just 14%, striking an ideal balance for predictive maintenance systems.

```
# Pipeline
pipeline = Pipeline([
    ("smote", SMOTE(random_state=42)),
    ("model", RandomForestClassifier(random_state=42))
])

# Parameter grid
param_grid = {
    'model__n_estimators': [300, 400, 500, 600, 700, 800],
    'model__max_depth': [None, 40],
    'model__min_samples_split': [2, 5],
    'model__min_samples_leaf': [1, 2],
    'model__max_features': ['sqrt'],
}

# Scoring metrics
scoring = {
    'f1': make_scorer(f1_score),
    'recall': make_scorer(recall_score)
}

# Stratified CV for imbalanced dataset
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Grid search
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring=scoring,
    refit='f1',
    cv=cv,
    n_jobs=-1,
    verbose=2
)

# Fit
grid_search.fit(X_train_processed, y_train)

# Best parameters
print("Best Parameters:", grid_search.best_params_)

# Get the best model from RandomizedSearchCV
best_rf = grid_search.best_estimator_

# Threshold-based prediction
# Get predicted probabilities for Faulty class (1)
y_proba = best_rf.predict_proba(X_test_processed)[: , 1]

# Set your threshold
threshold = 0.45

# Apply threshold to get final predictions
y_pred = (y_proba >= threshold).astype(int)
```

Regression

Feature Name	Description
House ID	A unique identifier assigned to each property.
City	The city where the property is located (e.g., Chicago, Denver, Seattle, New York).
House Area (sqm)	The total built-up area of the house measured in square meters (sqm) .
No. of Bedrooms	The total number of bedrooms available in the property.
No. of Toilets	The total number of toilets available in the property.
Stories	The number of floors or levels in the house.
Renovation Status	Indicates the furnishing condition — furnished , semi-furnished , or unfurnished .
Price (\$)	The target variable representing the sale price of the property in U.S. dollars.

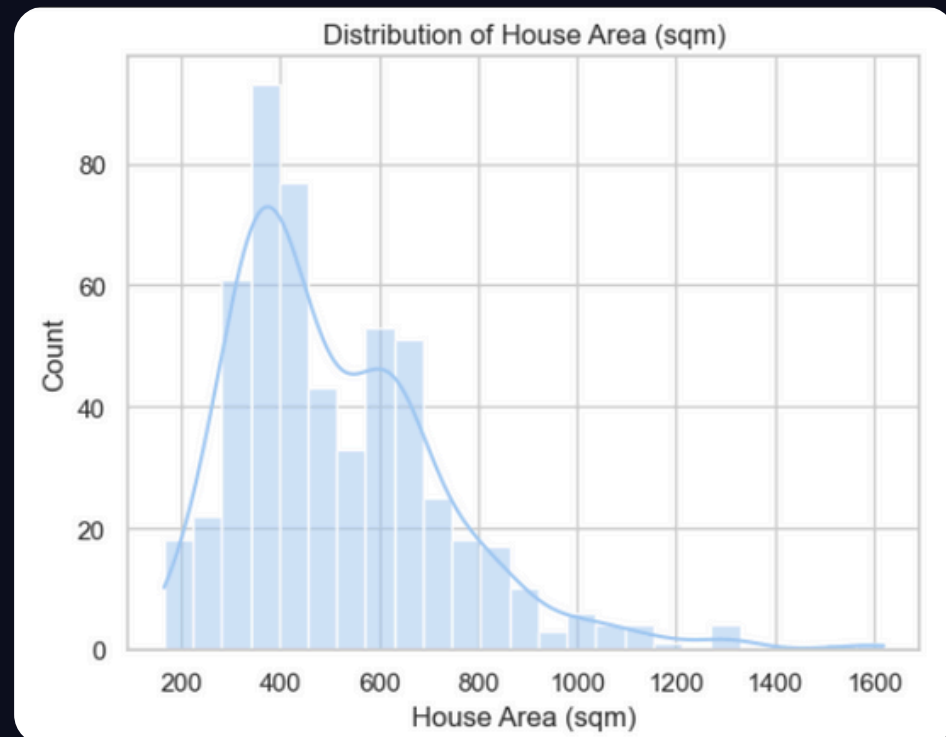
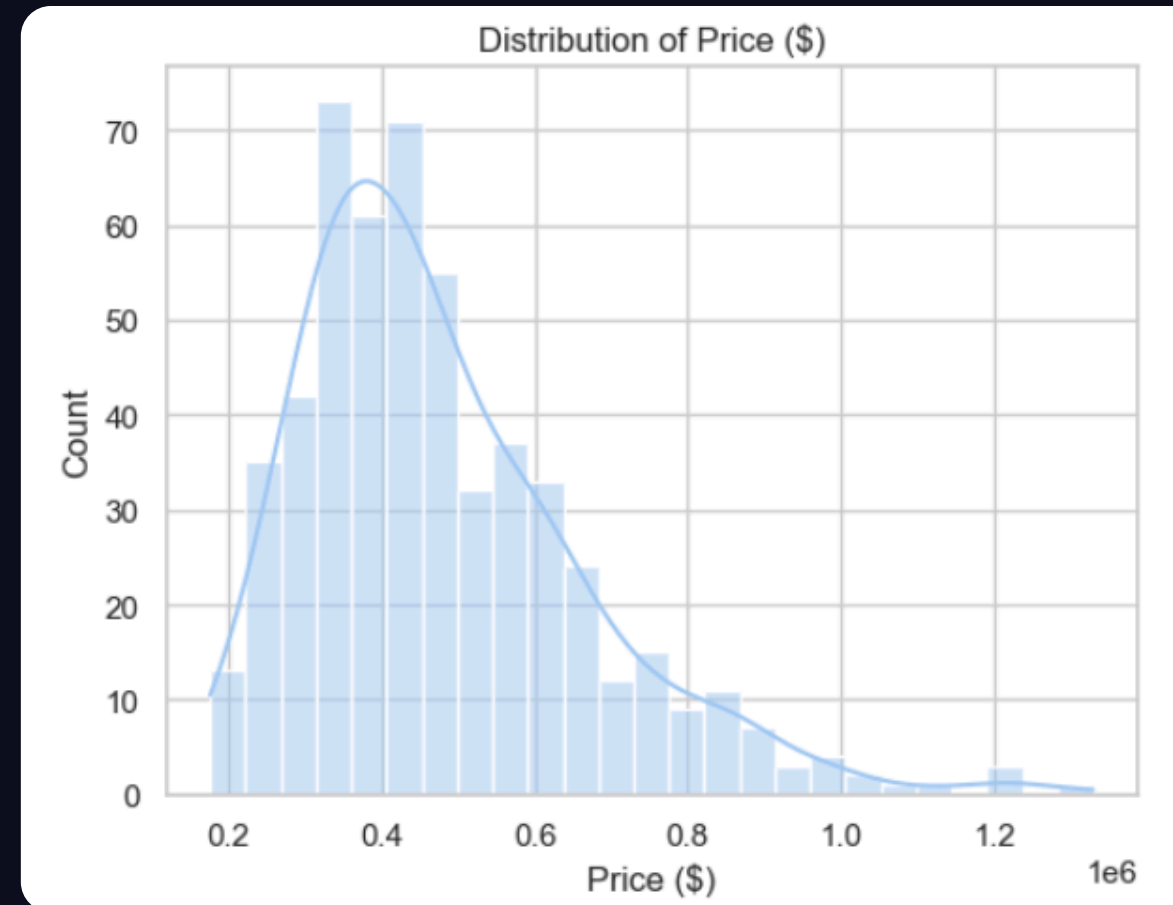
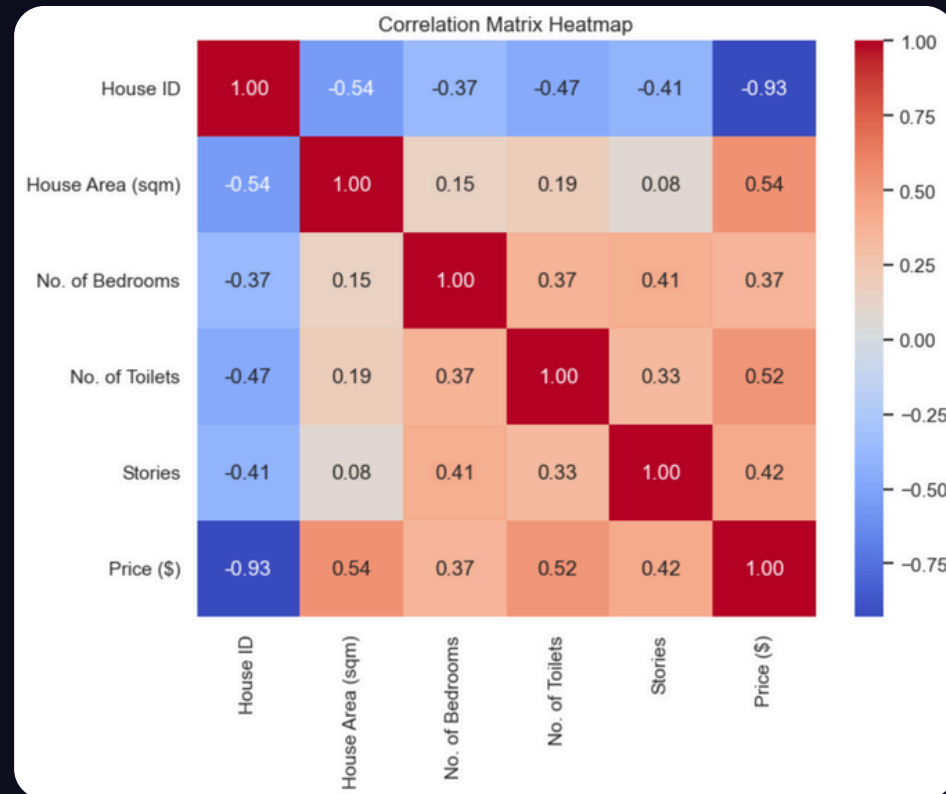
← Target Variable

Data Overview

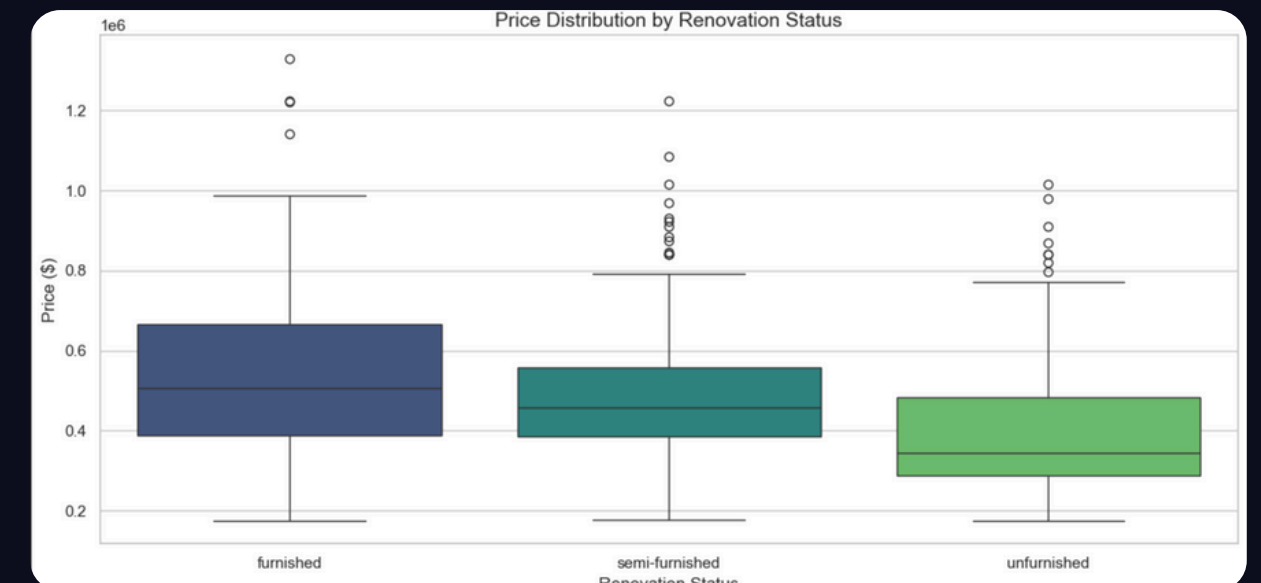
housing_price_data.csv contains **545** Rows and **8** Attributes, comprising as a mix of **Categorical** and **Numerical** features.

Objective: Build a **regression** model that accurately estimates housing prices.

EDA Highlights



- House Area shows the strongest positive correlation with Price
- Prices are right-skewed, with most homes in the lower range and a few high-value outliers.
- The distribution of house area mirrors the skew seen in price, further indicating a strong relationship between larger homes and higher prices.
- Price distributions vary significantly across cities, with differences in medians and outliers.
- Furnished and semi-furnished homes tend to have higher prices than unfurnished ones.



Data Preprocessing

- **Dropped** irrelevant identifier columns like House ID.
- Applied **Dummy-Encoding** to City and **Label-Encoding** to Renovation Status to convert categorical variables into numerical form.
- **Engineered** additional features such as the Toilets/Bedroom ratio and applied a **log transformation** to Area to reduce skewness.
- Separated the dataset into **Features (X)** and **Target (log-transformed Price)**.
- Performed a **stratified** 80-20 train-test-split to maintain proportional representation across categories.
- Scaled all numerical feature columns using **StandardScaler**, fitted on the training data to prevent data leakage.
- Left the target variable unscaled as it represents transformed price values.
- Verified final dataset shapes and ensured all processed data was complete and consistent.

```
# 1. Drop irrelevant columns
df = df.drop(['House ID'], axis=1)

# 2. Encode categorical variables
# Dummy encode 'City' and label encode 'Renovation Status'
df = pd.get_dummies(df, columns=['City'], drop_first=True)

dummy_cols = [col for col in df.columns if col.startswith("City_")]
df[dummy_cols] = df[dummy_cols].astype(int)

label_encoder = LabelEncoder()
df['Renovation Status'] = label_encoder.fit_transform(df['Renovation Status'].astype(str))

# 3. Feature Engineering
df['Toilets/Bedroom'] = df['No. of Toilets'] / df['No. of Bedrooms']
df['log_Area'] = np.log1p(df['House Area (sqm)']) # ->
df['Area_per_Bedroom'] = df['House Area (sqm)'] / df['No. of Bedrooms']
df['Size_Score'] = df['House Area (sqm)'] * df['Stories']
df['Bed_Toilet_Balance'] = df['No. of Bedrooms'] / df['No. of Toilets']
df['Total_Rooms'] = df['No. of Bedrooms'] + df['No. of Toilets']
df['Stories_Bedrooms'] = df['Stories'] * df['No. of Bedrooms']

# 4. Separate features and target variable
X = df.drop('Price ($)', axis=1) # Features
y = np.log1p(df['Price ($)']) # Target -> log1p adds 1 to prevent zero error

# 5. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 6. Feature Scaling ONLY numerical features
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns

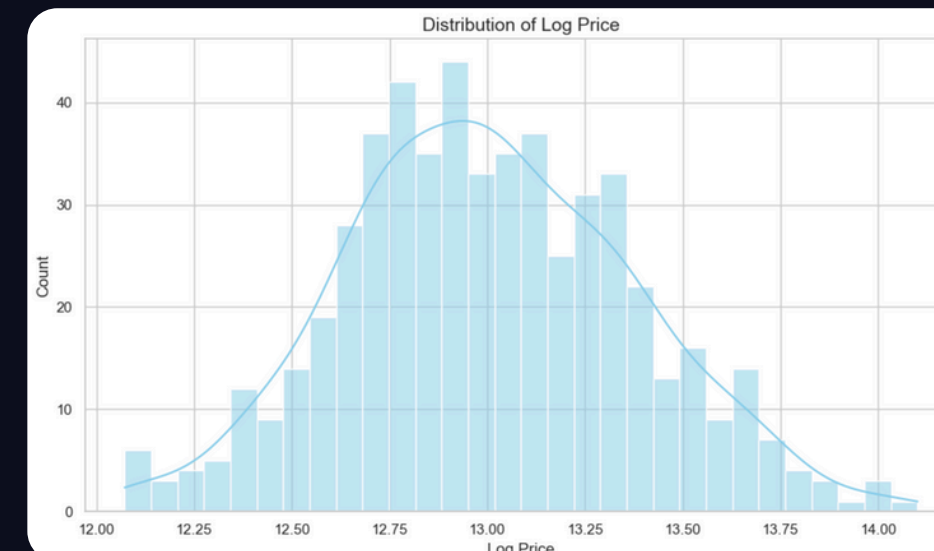
scaler = StandardScaler()
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])
```

	House Area (sqm)	No. of Bedrooms	No. of Toilets	Stories	Renovation Status	Price (\$)
0	742.0	4	2	3	0	1330000
1	896.0	4	4	4	0	1225000
2	996.0	3	2	2	1	1225000
3	750.0	4	2	2	0	1221500
4	742.0	4	1	2	0	1141000

City_Chicago	City_Denver	City_New York	City_Seattle
1	0	0	0
0	1	0	0
1	0	0	0
0	0	0	1
0	0	1	0

Toilets/Bedroom	log_Area	Area_per_Bedroom	Size_Score
0.500000	6.610696	185.5	2226.0
1.000000	6.799056	224.0	3584.0
0.666667	6.904751	332.0	1992.0
0.500000	6.621406	187.5	1500.0
0.250000	6.610696	185.5	1484.0

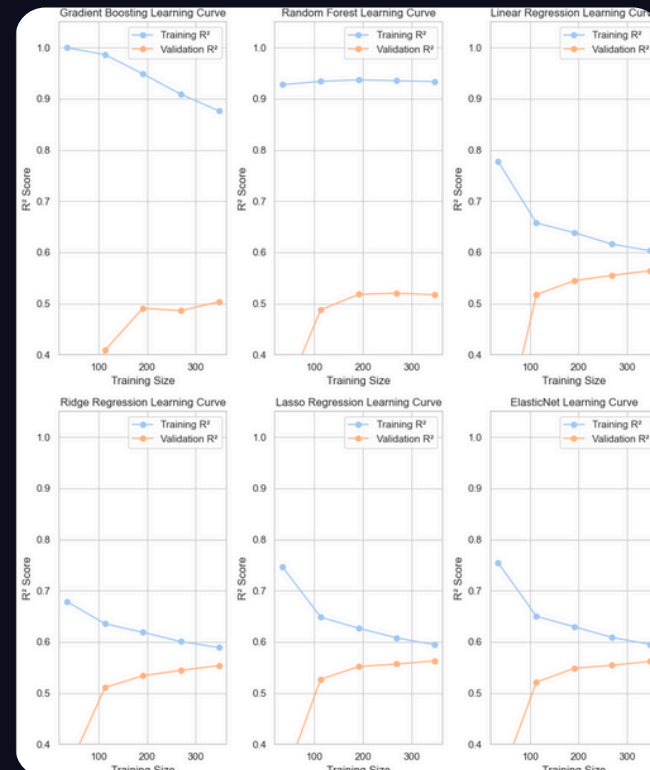
Bed_Toilet_Balance	Total_Rooms	Stories_Bedrooms
2.0	6	12
1.0	8	16
1.5	5	6
2.0	6	8
4.0	5	8



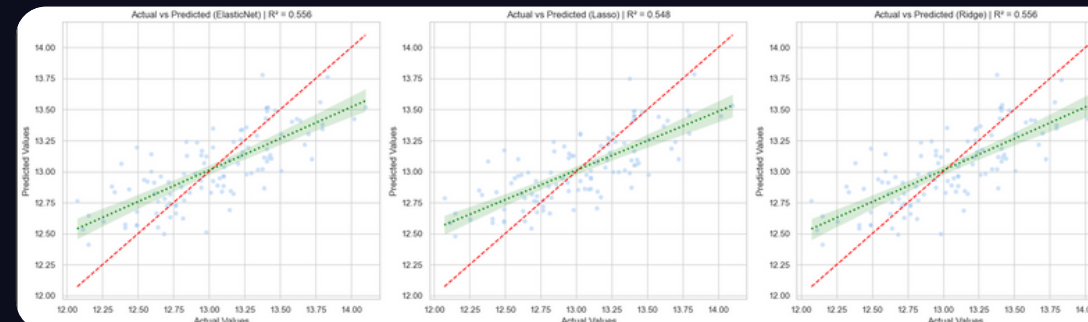
Training, Selection & Evaluation

Models chosen for Training

- Gradient Boosting Regressor
- Random Forest Regressor
- Linear Regression
- **Ridge Regression**
- **Lasso Regression**
- **ElasticNet Regression**



Gradient Boosting: Mean $R^2 = 0.5140$ | Std = 0.0338
Random Forest: Mean $R^2 = 0.5109$ | Std = 0.0356
Linear Regression: Mean $R^2 = 0.5632$ | Std = 0.0199
Ridge Regression: Mean $R^2 = 0.5640$ | Std = 0.0187
Lasso Regression: Mean $R^2 = 0.5654$ | Std = 0.0224
ElasticNet: Mean $R^2 = 0.5657$ | Std = 0.0188



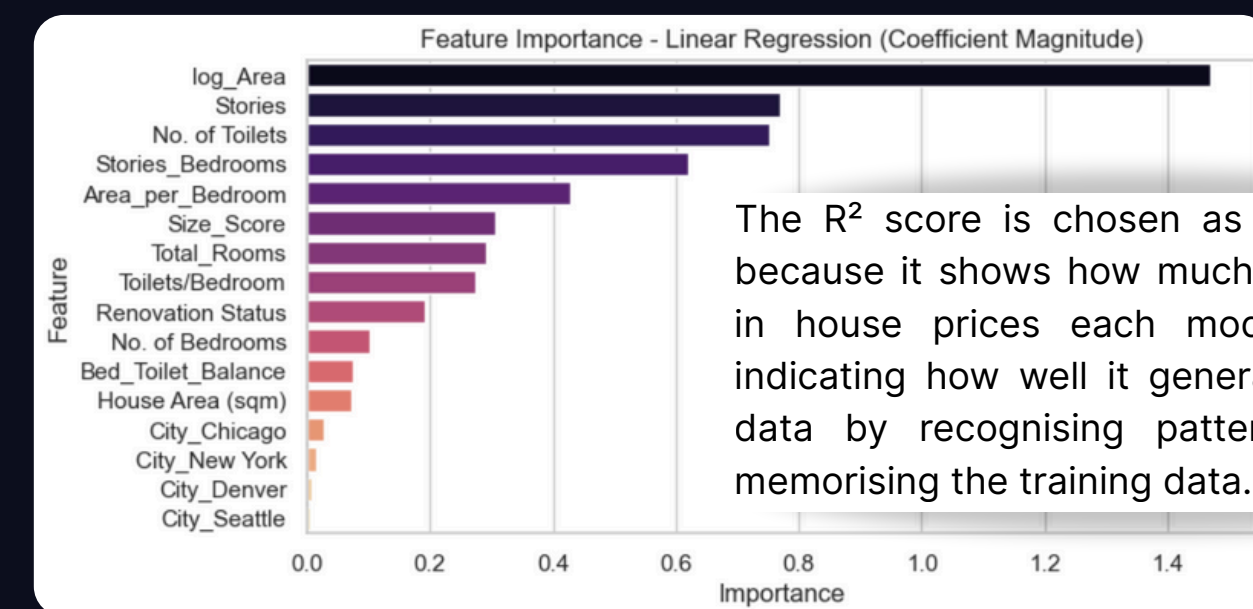
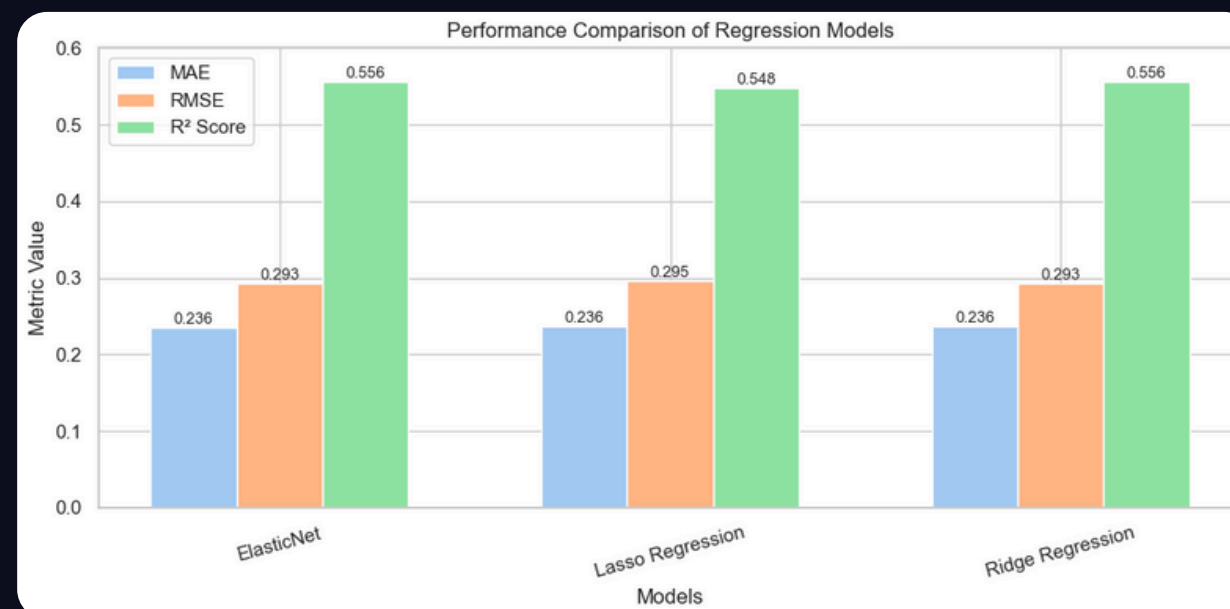
Best Model: ElasticNet Regression

- 5 Fold CV R^2 Score: **0.566** → Model explains 56.3% of the variance on average across folds.
- R^2 Score: **0.556** → Close to CV Score - no obvious overfitting/underfitting.
- RMSE: **0.293** → Predictions are off by ~0.293 units for log-transformed target.
- MAE: **0.236** → Since MAE < RMSE, large errors are not dominating models mistakes.

Training Results



1. **ElasticNet Regression**
2. Lasso Regression
3. ElasticNet Regression



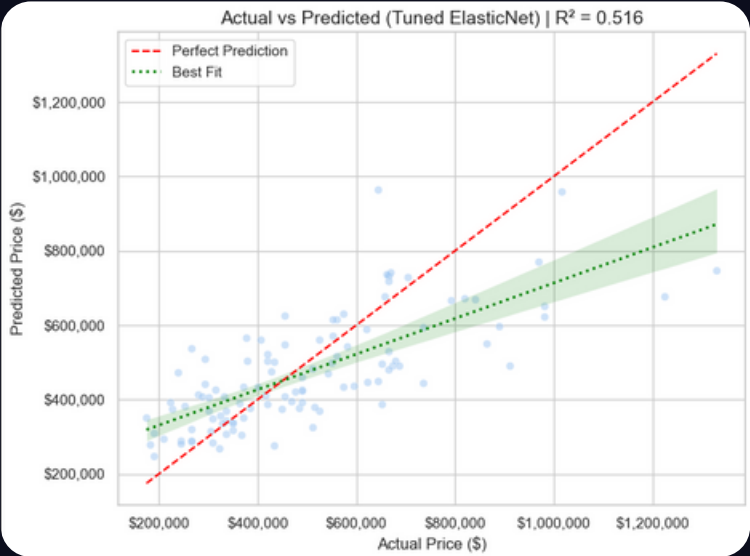
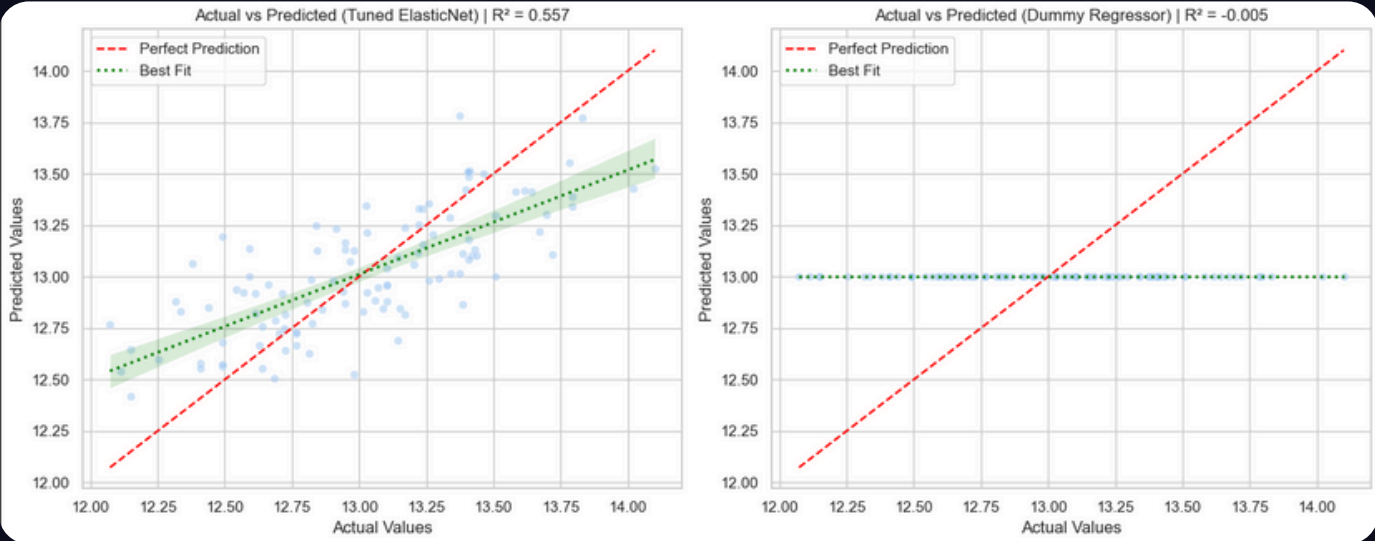
The R^2 score is chosen as the best metric because it shows how much of the **variance** in house prices each model can explain, indicating how well it generalises to unseen data by recognising patterns rather than memorising the training data.

Model Improvement

Metric	Value	Interpretation
R ² Score	0.516	Model explains 51.6% of price variance
MAE	\$115,536	Average prediction error in absolute terms
RMSE	\$156,387	Square root of mean squared error (penalizes large errors more)
Prediction Range	\$200K - \$1.2M	Model handles diverse price ranges
Visual Fit	Strong positive correlation	Predictions align well with actual values

ElasticNet Tuning Summary

- GridSearchCV (10-fold CV) used to optimise ElasticNet for house-price prediction
- **Parameters searched:**
 - alpha: 15 values (0.0001 → 10)
 - l1_ratio: 0.1 to 1.0
- **Best hyperparameters:**
 - alpha = 0.001178 (very light regularization)
 - l1_ratio = 0.7 (more L1 than L2)
- **Performance:**
 - Best CV R²: 0.548
 - Test R²: 0.557
 - MAE: 0.236
 - RMSE: 0.293
- **Key takeaways:**
 - Only slight improvement over baseline
 - Data prefers minimal regularization
 - L1 emphasis suggests some feature selection benefits
 - Model still limited by linear assumptions



```
# Create pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('elasticnet', ElasticNet(max_iter=10000, random_state=42))
])

# Define parameter grid
param_grid = {
    'elasticnet__alpha': np.logspace(-4, 1, 15),
    'elasticnet__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 1.0]
}

# Grid search
grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=10,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
```

In conclusion, the final optimised model achieves good performance, explaining 55.7% of variance and predicting house prices within \$156k on average, providing reliable estimates for real estate valuation in this market.

CV R²: 0.548
Test R²: 0.557